

From Trading Idea to MT5 Indicator

A Practical, Step-by-Step Guide to Turning a Trading Idea Into Working MT5 Code Using AI

Introduction

You do not need to be a programmer to start automating your trading strategy.

The first step is not writing code. The first step is explaining what you want to build.

AI can help you take a simple trading idea and turn it into a clear specification that a coding AI can understand. The process is simple:

IDEA !' SPECIFICATION !' CODING PROMPT !' CODE !' MT5 !' TEST !' IMPROVE

In this guide we use one worked example — an MT5 Average Daily Range (ADR) indicator — to demonstrate the full process from idea to finished tool. The goal here is not to teach you how ADR works as a trading concept. The goal is to show you how to take an idea from your head and use AI to turn it into a working MT5 indicator running live on an MT5 chart.

Part 1 — Start With the Idea

When you have an idea for an indicator or Expert Advisor, don't start by thinking about programming. Start by asking:

“What do I want this tool to do?”
STEP 1 — YOUR ROUGH IDEA (GIVEN TO CHATGPT)

Create specifications for me to tell AI to build an MT5 ADR indicator that is based on X number of days. It's calculated from the opening price of the day to the high or low of the day, and it should draw dotted lines every day after market open, and when it's reached, it should give a notification, and it should also show the number of days used. I want a notification on screen when reached, and it should show the number of pips to reach the level. Make it clear and simple to understand.

Part 2 — Let ChatGPT Help Structure the Idea

Now ChatGPT's job is to take your rough idea and turn it into a clearer specification. It can help identify things such as:

- What information the indicator needs from you.
- What it should calculate.
- What should appear on the chart.
- What should happen when a level is reached.
- What notifications are required.
- What information should be displayed to you.

This is where AI becomes your specification assistant. You are not asking it to decide your trading strategy — you are asking it to help you clearly describe your strategy or idea.

Part 3 & 4 — Review, Accept, Modify or Delete

This is one of the most important parts of the process. Do not automatically accept everything AI gives you. Read through the specification carefully and ask yourself:

“Is this actually what I want?”

When reviewing the AI response, you have three simple choices:

1. Accept: If AI understood your idea correctly, keep it.
2. Modify: If something is almost correct but needs to change, tell AI (e.g., "Change this so indicator uses 10 days instead of 5").
3. Delete: If AI added something you don't want, remove it (e.g., "Remove the historical chart display. I only want the current day's levels").

Part 5 & 6 — Finalize Specification & Coding Prompt

Once you have reviewed everything, you have a final specification. This becomes the blueprint for your indicator.

Now move to the next stage. Instead of taking your original rough request directly to a coding AI, use ChatGPT to prepare a proper coding prompt for Claude:

Prompt: "Now take this finalized specification and create a detailed coding prompt for Claude to build this MT5 indicator in MQL5."

The 16-Step AI Automation Workflow

1. Idea: You have a trading idea.
2. ChatGPT: Explain the idea in normal language.
3. Brainstorm: ChatGPT helps turn idea into structured specification.
4. Review: Read what AI created.
5. Accept / Modify / Delete: Keep what is correct, remove what you don't want.
6. Finalize: Create the final specification.
7. Coding Prompt: Ask ChatGPT to turn specification into prompt for Claude.
8. Review Prompt: Ensure Claude will understand exactly what to build.
9. Claude: Give the prompt to Claude.
10. MQL5 Code: Claude generates the code (see Appendix A).
11. MT5 / MetaEditor: Put the code into MetaEditor.
12. Compile: Check whether the code compiles.
13. Debug: If errors, send exact compiler error text back to Claude.
14. Test: Run indicator on live/demo chart in MT5.
15. Validate: Compare actual behaviour with original specification.
16. Improve: Make changes where necessary.

Three Golden Rules

Rule 1 — Idea First, Code Last

Never start by asking AI to write code when you haven't clearly defined what you want. Start with the idea. Turn the idea into a specification. Then create the code.

Rule 2 — Don't Blindly Trust AI

AI can misunderstand you. AI can make assumptions. AI can add things you never requested. Always review the output. You are the decision-maker.

Rule 3 — Test the Behaviour

A green compilation message is not proof that your strategy has been automated correctly. Always test the actual result against your original idea.

Appendix A — Complete MQL5 Source Code: ADR_Levels.mq5

This is the complete MQL5 code generated for the ADR indicator case study. Copy and paste into MetaEditor as a Custom Indicator:

Source Code Listing: ADR_Levels.mq5

```
//+-----+
//|                                     ADR_Levels.mq5 |
//|                                     Case Study: AI-Assisted Trading Automation |
//|                                     Average Daily Range (ADR) breakout-level indicator for MT5 |
//+-----+
#property copyright "Case Study - AI-Assisted Trading Automation"
#property link      ""
#property version   "1.00"
#property indicator_chart_window
#property indicator_plots 0

//--- Input parameters -----
input group "ADR Settings"
input int   InpADRDays      = 4;      // Number of previous completed days used for ADR
input double InpBufferPoints = 0;      // Extra buffer added to each level (points)
input bool   InpShowDailyOpen = true;  // Draw a line at today's opening price

input group "Notifications"
input bool   InpAlertPopup   = true;   // Show a terminal Alert() popup
input bool   InpAlertPush    = false;  // Send a push notification to mobile
input bool   InpAlertSound    = true;   // Play a sound when a level is reached
input string InpAlertSoundFile = "alert.wav";

input group "Appearance"
input color   InpColorHigh    = clrDeepSkyBlue;
input color   InpColorLow     = clrOrangeRed;
input color   InpColorOpen    = clrGray;
input color   InpColorHitHigh = clrLime;
input color   InpColorHitLow  = clrRed;
input int     InpFontSize     = 9;
input int     InpLineWidth    = 1;

//--- Internal state -----
double   g_adrHighLevel = 0.0;
double   g_adrLowLevel  = 0.0;
double   g_todayOpen    = 0.0;
datetime g_todayStart   = 0;
bool     g_highReached  = false;
bool     g_lowReached   = false;
double   g_pointFactor  = 1.0;
string   g_prefix;

int OnInit()
{
    g_prefix = "ADR_" + _Symbol + "_";
    g_pointFactor = (_Digits == 3 || _Digits == 5) ? 10.0 : 1.0;
    CalculateADR();
    DrawLevels();
    return(INIT_SUCCEEDED);
}

void OnDeinit(const int reason)
{
    ObjectsDeleteAll(0, g_prefix);
    Comment("");
}

int OnCalculate(const int rates_total,
                const int prev_calculated,
                const datetime &time[],
                const double &open[],
                const double &high[],
                const double &low[],
                const double &close[],
                const long &tick_volume[],
                const long &volume[],
                const int &spread[])
{
    datetime barDayStart = iTime(_Symbol, PERIOD_D1, 0);
    if(barDayStart != g_todayStart)
    {
        g_todayStart = barDayStart;
        g_highReached = false;
        g_lowReached  = false;
        CalculateADR();
        DrawLevels();
    }
    UpdateLiveInfo();
    return(rates_total);
}
```

```

void CalculateADR()
{
    double sumUp = 0.0, sumDown = 0.0;
    int counted = 0;
    for(int i = 1; i <= InpADRDays; i++)
    {
        double dOpen = iOpen(_Symbol, PERIOD_D1, i);
        double dHigh = iHigh(_Symbol, PERIOD_D1, i);
        double dLow = iLow(_Symbol, PERIOD_D1, i);
        if(dOpen == 0.0) continue;
        sumUp += (dHigh - dOpen);
        sumDown += (dOpen - dLow);
        counted++;
    }
    if(counted == 0) return;
    double avgUp = sumUp / counted;
    double avgDown = sumDown / counted;
    double buffer = InpBufferPoints * _Point;

    g_todayOpen = iOpen(_Symbol, PERIOD_D1, 0);
    g_adrHighLevel = g_todayOpen + avgUp + buffer;
    g_adrLowLevel = g_todayOpen - avgDown - buffer;
}

void DrawLevels()
{
    datetime startTime = g_todayStart;
    datetime endTime = startTime + PeriodSeconds(PERIOD_D1) * 2;
    DrawDottedLine(g_prefix + "High", startTime, endTime, g_adrHighLevel, InpColorHigh);
    DrawDottedLine(g_prefix + "Low", startTime, endTime, g_adrLowLevel, InpColorLow);
    if(InpShowDailyOpen)
        DrawDottedLine(g_prefix + "Open", startTime, endTime, g_todayOpen, InpColorOpen);
    UpdateLabel(g_prefix + "HighLabel", g_adrHighLevel, InpColorHigh,
        StringFormat("ADR HIGH (%d-day) %s", InpADRDays, DoubleToString(g_adrHighLevel, _Digits)));
    UpdateLabel(g_prefix + "LowLabel", g_adrLowLevel, InpColorLow,
        StringFormat("ADR LOW (%d-day) %s", InpADRDays, DoubleToString(g_adrLowLevel, _Digits)));
}

void DrawDottedLine(string name, datetime t1, datetime t2, double price, color clr)
{
    if(ObjectFind(0, name) < 0)
        ObjectCreate(0, name, OBJ_TREND, 0, t1, price, t2, price);
    else
        ObjectMove(0, name, 0, t1, price);
    ObjectSetInteger(0, name, OBJPROP_TIME, 1, t2);
    ObjectSetDouble(0, name, OBJPROP_PRICE, 1, price);
    ObjectSetInteger(0, name, OBJPROP_COLOR, clr);
    ObjectSetInteger(0, name, OBJPROP_STYLE, STYLE_DOT);
    ObjectSetInteger(0, name, OBJPROP_WIDTH, InpLineWidth);
    ObjectSetInteger(0, name, OBJPROP_RAY_RIGHT, false);
    ObjectSetInteger(0, name, OBJPROP_SELECTABLE, false);
    ObjectSetInteger(0, name, OBJPROP_HIDDEN, true);
}

void UpdateLabel(string name, double price, color clr, string text)
{
    if(ObjectFind(0, name) < 0)
        ObjectCreate(0, name, OBJ_TEXT, 0, TimeCurrent(), price);
    ObjectSetString(0, name, OBJPROP_TEXT, text);
    ObjectSetInteger(0, name, OBJPROP_TIME, TimeCurrent());
    ObjectSetDouble(0, name, OBJPROP_PRICE, price);
    ObjectSetInteger(0, name, OBJPROP_COLOR, clr);
    ObjectSetInteger(0, name, OBJPROP_FONTSIZE, InpFontSize);
    ObjectSetInteger(0, name, OBJPROP_ANCHOR, ANCHOR_LEFT_LOWER);
    ObjectSetInteger(0, name, OBJPROP_SELECTABLE, false);
    ObjectSetInteger(0, name, OBJPROP_HIDDEN, true);
}

void UpdateLiveInfo()
{
    double bid = SymbolInfoDouble(_Symbol, SYMBOL_BID);
    double pipsToHigh = (g_adrHighLevel - bid) / _Point / g_pointFactor;
    double pipsToLow = (bid - g_adrLowLevel) / _Point / g_pointFactor;

    if(!g_highReached && bid >= g_adrHighLevel)
    {
        g_highReached = true;
        NotifyLevelReached("ADR HIGH", g_adrHighLevel);
        UpdateLabel(g_prefix + "HighLabel", g_adrHighLevel, InpColorHitHigh,
            StringFormat("ADR HIGH +%s pts - REACHED", DoubleToString((g_adrHighLevel - g_todayOpen) / _Point,
1)));
    }
}

```

```

if(!g_lowReached && bid <= g_adrLowLevel)
{
    g_lowReached = true;
    NotifyLevelReached("ADR LOW", g_adrLowLevel);
    UpdateLabel(g_prefix + "LowLabel", g_adrLowLevel, InpColorHitLow,
        StringFormat("ADR LOW -%s pts - REACHED", DoubleToString((g_todayOpen - g_adrLowLevel) / _Point,
1)));
}

string panel = StringFormat(
    "ADR (%d-Day) Indicator\n" +
    "Daily Open : %s\n" +
    "ADR High   : %s (%s)\n" +
    "ADR Low    : %s (%s)\n" +
    "Pips to High: %s\n" +
    "Pips to Low : %s",
    InpADRDays,
    DoubleToString(g_todayOpen, _Digits),
    DoubleToString(g_adrHighLevel, _Digits), (g_highReached ? "REACHED" : "pending"),
    DoubleToString(g_adrLowLevel, _Digits), (g_lowReached ? "REACHED" : "pending"),
    (g_highReached ? "-" : DoubleToString(MathMax(pipsToHigh, 0), 1)),
    (g_lowReached ? "-" : DoubleToString(MathMax(pipsToLow, 0), 1))
);
Comment(panel);
}

void NotifyLevelReached(string levelName, double levelPrice)
{
    string msg = StringFormat("%s: %s level reached at %s (%s)",
        _Symbol, levelName, DoubleToString(levelPrice, _Digits),
        TimeToString(TimeCurrent(), TIME_DATE | TIME_MINUTES));
    if(InpAlertPopup) Alert(msg);
    if(InpAlertPush) SendNotification(msg);
    if(InpAlertSound) PlaySound(InpAlertSoundFile);
    Print(msg);
}

```


